arXiv:2203.07589v2 [cs.RO] 3 May 2022

受落脚点约束的双足动态行走的仿真到现实学习

Helei Duan, Ashish Malik, Jeremy Dao, Aseem Saxena, Kevin Green, Jonah Siekmann†, Alan Fern, Jonathan Hurst

摘要——近年来，针对双足机器人的强化学习（***RL***）研究已成功学习到多种动态步态的控制器，并展示了鲁棒的仿真到现实（***sim-to-real***）效果。为了保持平衡，所学控制器在落脚点位置选择上拥有完全的自由度，从而产生高度鲁棒的步态。然而，在现实世界中，环境通常会对可行的落脚点位置施加约束，这些约束通常由感知系统识别。遗憾的是，目前大多数在双足机器人上演示的***RL***控制器都无法指定并响应此类约束。这一控制接口的缺失极大地限制了当前***RL***控制器的实际应用。本文旨在保持所学步态的鲁棒性与动态性，同时满足外部施加的落脚点约束。我们提出了一种强化学习方案，用于训练能够响应指定触地位置的动态步态控制器。随后，我们在双足机器人***Cassie***上成功验证了其仿真性能及仿真到现实的迁移性能。此外，我们利用监督学习归纳出一个转移模型，以根据机器人的本体感受观测准确预测控制器能够实现的下一个落脚点位置。该模型为将所学控制器集成到全阶机器人运动规划器中铺平了道路，使其能够鲁棒地同时满足平衡约束和环境约束。

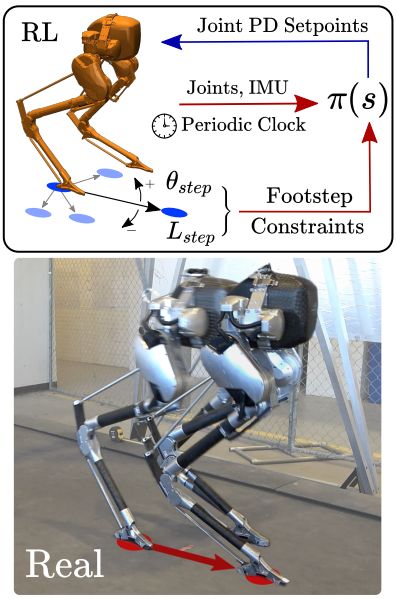


图1：我们旨在探讨如何在要求机器人满足特定脚步约束的同时，保持来自学习方法的动态步态。图中显示Cassie从原地踏步开始，执行一次受控的1-2步动作（向前半米步）以满足给定的步幅指令，然后恢复动态平衡。以往许多基于RL的步态策略不具备这种控制权限，而是允许脚步完全自由。

I. 引言

强化学习（RL）在双足机器人中的应用近期已取得成功，能够学习到可在多种敏捷步态（如行走、跑步、跳跃和蹦跳）之间切换的控制器，并实现了稳健的仿真到现实迁移 [1–3]。双足动物同样会表现出不同步态之间的动态切换，但此外它们还能对具体落脚点进行精确控制，这些落脚点可能偏离自然步态，以穿越离散地形。相比之下，大多数强化学习方法尚未支持能够考虑落脚点约束的控制器，而是允许控制器任意选择落脚点位置以维持平衡。这极大地限制了现有强化学习控制器在具有不可踩踏区域（如坑洞）或需要精确落脚位置的真实世界应用中的适用性。

在这项工作中，我们探索将强化学习（RL）用于双足控制器，使其既能稳健地保持平衡，又能动态满足作为额外控制输入提供的特定落脚点约束。理想情况下，学习到的控制器应能满足实时落脚点约束，当这些约束在动态上可行时，并应在约束可能导致不稳定或摔倒时优先保持平衡。我们的方法扩展了先前关于周期性步态强化学习的研究 [2]，将落脚点约束纳入奖励函数，并在仿真中针对随机化的落脚点约束进行训练。由此得到的步态控制器既能通过提供恒定步长约束来执行周期性步态，也能在机器人能力范围内跟随变化的步长和方向指令序列。

我们在双足机器人 Cassie 上通过仿真和硬件实验演示了这种方法，如图 1 所示。学习得到的控制器对变化的约束序列、外部扰动和不平地形表现出鲁棒性，同时通常能实现落脚点目标。此外，为了支持该控制器用于更高层的运动规划，我们探索了有监督学习，以构建一个从触地到下一次触地（TD2TD）的转移模型；该模型仅依据可观测的机器人状态即可预测可达的触地位置集合。结果表明，可以学习到精确的 TD2TD 模型，从而为未来的全身多步前瞻模型预测规划研究奠定基础。

*本工作由美国国家科学基金会（NSF）资助，项目编号 IIS-1849343、DGE-1314109，以及DARPA合同 W911NF-16-1-0002。所有作者均隶属于俄勒冈州立大学协作机器人与智能系统研究所，美国俄勒冈州科瓦利斯市 97331。电子邮件：{duanh, malikas, daoje, saxenaa, greenkev, siekmanj, afern, jhurst@oregonstate.edu}。

†同时任职于Agility Robotics（美国俄勒冈州奥尔巴尼市，邮编97321）。

II. 相关工作

面向双足机器人的仿真到现实强化学习。基于学习的控制方法已在真实双足机器人上展现了令人印象深刻的运动能力。一种常见做法是奖励对模板动作库 [1, 4–7] 的模仿。另一种方法是设计具有物理原理依据的奖励函数 [2]，并将其与一组行为参数（如每条腿的摆动/支撑比）耦合。两种方法都已在仿真到现实迁移中展示出成功的演示效果。这些学习到的控制器甚至能在没有感知信息的情况下成功穿越多段楼梯 [8]。为了使学习到的解决方案能够实际用于双足机器人，学习到的策略应当有能力遵守来自外部环境的落脚点约束；这在理论上将提供对运动行为的更多控制，从而使机器人能够在现实世界的约束下执行任务。

足落点约束的腿足机器人学习。腿足机器人具有独特优势，能够动态选择稀疏的足底支撑点，以克服具有挑战性的地形。此前有多项研究聚焦于使用学习方法，使机器人到达特定的目标足底触地位置。一些基于学习的方法通常会在学习任何运动行为之前，利用已知模型或启发式规则来分配目标触地位置 [9, 10],，而另一些方法则训练能够最大化跟踪目标足落点的步态策略 [4, 11]。尤其是，我们的工作受到 Xie 等人 [11]，的启发，他们在仿真中通过课程学习引导预训练的运动策略来跟踪目标足落点。相比之下，我们的方法从头开始训练步态策略，且不使用课程学习来最大化性能。尽管如此，我们仍实现了成功的仿真到现实迁移，这可能得益于基于物理原理的周期性奖励设计。

腿式机器人转移模型学习。对双足运动的研究在很大程度上依赖于将步态参数化为简化的表征，并配合使用降阶动力学模型。其中许多方法根据步态事件（如触地、足尖离地或腾空阶段的最高点）来定义控制动作。这些紧凑模型 [12–14] 被广泛用于双足机器人的控制与规划框架中，以决定下一步的落脚点或未来的位置与力轨迹 [15–17]。虽然它们能够捕捉运动的核心欠驱动动力学，但降阶模型与全阶机器人之间仍然存在显著差距，例如运动学和惯性属性上的不匹配。已有研究致力于在降阶模型之上引入额外的学习过程来改进这些模型[18–20]。另一种方法是从底层自下而上地构建或精化转移模型，通过从一些可工作的控制器中获取数据 [21–23]。在给定全阶机器人控制策略的情况下，我们寻求利用神经网络学习一个从机器人本体感受状态到下一次触地位置可达性的映射。因此，借助这一学习得到的转移模型，规划将变得可行。

III. 学习足落点约束的步态策略

我们提出的带落脚点约束的学习框架以先前双足行走工作中的强化学习框架为基础，并对其进行了扩展 [2]。如下所述，与先前工作的一个关键区别在于，我们的控制策略接收的命令输入约束的是相对落脚点（见图2），而非约束速度和周期性步态特征。因此，这一区别要求调整奖励设定，以鼓励匹配落脚点命令，而非周期性步态特性。

A. 状态空间

策略的状态空间由观测到的机器人状态和用户指令构成，其大小为 $S \in \mathbb{R}^{42}$。更具体地说，S 包括本体的旋转角速度（$\mathbb{R}^3$）、以四元数表示的姿态（$\mathbb{R}^4$），以及在硬件上可测量的驱动与非驱动关节的位置和速度（$\mathbb{R}^{28}$）。S 还包含一个二维周期时钟输入，该输入由两个具有相位偏移的正弦函数计算得出，以生成唯一的相位标识并实现周期循环（时钟细节参见先前工作[2] ）。状态中还包含指令输入，分别表示期望的摆动-支撑比（$\mathbb{R}^2$）以及一个允许机器人偏移绝对角度以朝向前方的转向速率输入（$\mathbb{R}^1$）。状态的最后一个指令输入分量指定了期望的落脚点约束 U。具体而言，该指令指定当前支撑脚位置与同一只脚的期望下一个触地（TD）位置之间在极坐标（$\mathbb{R}^2$）下的相对偏移，其定义为 $U = (L_{step}, \theta_{step})$。

B. 动作空间

动作空间 $A \in \mathbb{R}^{10}$ 被定义为每个电机的 PD 设定点。动作由策略以 40Hz 的频率更新。底层关节 PD 控制器接收策略动作，并以 2kHz 的频率运行固定增益的关节 PD 控制。

C. 奖励

为了训练机器人既能保持平衡行走，又能满足落脚点约束，我们使用了两个不同时间尺度的奖励。平衡行走奖励是密集奖励，在每个决策周期都会产生；而落脚点约束奖励是稀疏奖励，仅在 TD 时刻产生。

该密集奖励的主要目标是鼓励每只脚按照时钟定义表现出摆动相与支撑相行为。此外，该密集奖励通过检查策略更新之间走过的距离，促使机器人向新的落脚点目标移动。该项计算为$r_{pelvis} = 100(\Delta d^i - \Delta d^{i-1})$，其中 d^i 是第 i 个策略步时目标到骨盆的距离。如果步距小于 0.1m，该奖励项则会转而通过惩罚较大的速度 $r_{pelvis} = 1 - (\exp(-3|\dot{x}|) + \exp(-3|\dot{y}|))$ 来鼓励稳定的骨盆运动。

在每个由时钟输入指示的 TD 事件中，稀疏奖励根据实际与目标脚步位置之间的欧几里得距离 f 计算为 $r_{step} = \exp(-2f)$。它将在每个脚步时被叠加到稠密奖励之上。

Although the sparse bonus could become a dense one by adding it throughout the entire swing phase, we found this will lead to aggressive and physically infeasible swing behaviors. The remaining reward terms listed in Table I, such as orientation and action smoothness, are defined the same as in [2], which we refer readers to for detailed reward definitions.

Name	Weight	Name	Weight
Right foot force	0.155	Left foot force	0.155
Right foot speed	0.125	Left foot speed	0.125
Robot orientation	0.125	Pelvis stability	0.125
Action smoothness	0.0325	Torque smoothness	0.0325
Pelvis	**0.125**	**Sparse step reward**	**3.0**

TABLE I: Reward Terms and Weights.

D. Policy Commands Generation

The nature of the reward structure introduced in [2] provides a clock for timing the swing and stance phases of the gait. When the robot learns how to swing the leg, it could potentially explore swinging towards the desired step target and would be rewarded for doing so. We use the clock values for each leg to infer the touchdown event by checking against a threshold value. Once the clock value passes the threshold and indicates a touchdown, we generate a new footstep command relative to the current stance foot. We do not use forces to check TD events in order to generate the new command or compute sparse reward to avoid explicit force sensing on hardware. The footstep target used for the reward function is subsequently computed in global coordinates by adding the command with the current stance foot location as shown in Figure 2. The policy input is also updated with the generated command, expressed as an offset from the last commanded touchdown position in polar coordinates. As a result, the policy has a one-step preview for the same side footstep target.

Each roll-out will be terminated if the robot falls by checking for large orientation changes. We do not use hard termination conditions when the robot fails to reach the target for a particular step. In other words, the robot can keep collecting rewards even if the touchdown locations are far from the desired ones. Since the footstep target generation is aligned with the periodic clock, the policy is encouraged to sync up its gait with the swing footstep targets in order to maximize the reward.

From biomechanics observations, bipedal locomotion can be viewed as a constrained optimization problem among traveling speed, step frequency, and step length [24]. Here we do not focus on tracking any commanded speed, but rather emphasize the step frequency and commanded footstep targets. Thus, step frequency becomes a critical control parameter and majorly affects the swing timing. The swing time influences how the robot can reach towards the next footstep target within the actuator's capability.

The periodic clocks cover a range of swing-to-stance ratios and step frequencies. The clock is represented with a cyclic

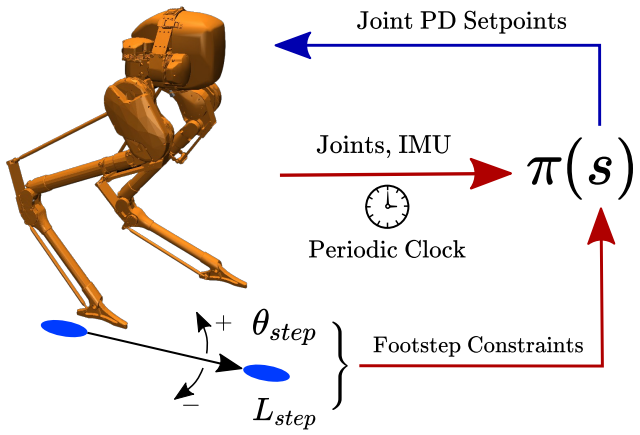


Fig. 2: The footstep-constrained gait policy maps proprioceptive information of joints and IMU sensors into joint PD setpoints. Importantly, the control policy is conditioned on footstep commands that are relative to the previous touchdown location for the same foot. The commands are updated at a random TD event and parameterized in the polar coordinate $(L_{step}, \theta_{step})$. Positive θ_{step} means the target touchdown location is left from the previous TD and vice versa. Although the policy uses polar coordinate for commands, the data representations are all plotted in square for convenience.

phase variable ϕ, and is computed as, $\phi^{i+1} = \phi^i + \gamma\Delta\phi^i$, where i is the policy update step, $\Delta\phi$ is the nominal increment, and γ is the phase multiplier that controls the scale of the phase increment, resulting in different step frequency.

Instead of having a constant γ, we randomize different combinations of ratio and step frequency, asking the policy to try to gather higher rewards by exploring the dynamics as much as possible. During each roll-out, to allow for moderate randomization, there is a 1/50 chance of randomizing the control parameters (swing ratio, step frequency, and footstep commands) at each policy step. Note that the footstep commands may be randomized at every policy step, but are only updated per TD event. All control parameters are randomized with uniform sampling from a set of ranges shown in Table II,

Control Parameter	Unit	Range
Swing-to-Stance Ratio	–	$[0.45, 0.6]$
Step Frequency γ	–	$[0.9, 1.3]$
Footstep Length L_{step}	m	$[0, 0.8]$
Footstep Direction θ_{step}	rad	$[-\pi, \pi]$

TABLE II: Control parameters are randomized during training.

E. Learning Algorithm and Setup

We use the Proximal Policy Optimization (PPO) algorithm [25], a model-free policy gradient method with an actor-critic network. Both the actor and critic use LSTM neural networks with two recurrent hidden layers of size 128. Actor and critic networks are independent of each other and do not share weights between them. We used the clipped objective for PPO based on the KL divergence between the updated policy and the previous policy. Following [5], we also deploy dynamics randomization during training.

IV. Footstep Behavior Prediction with Learned Transition Model

With the learned footstep-constrained gait policy, we aim to understand how predictable the next touchdown is by introducing a predictive model. This model tries to capture the TD2TD transition dynamics assuming the robot is controlled under the policy. Specifically, this model maps the robot states into the predicted step error, the Euclidean difference between the desired and actual footstep locations. This model spans the 2D command space, quantifying the step commands from the current state.

A. Data Collection

We begin with data collection given a trained footstep policy. The intention is to capture the 1-step behavior right after the robot takes a new step command. First, the 2D step command space is discretized into a 30×30 grid in terms of Cartesian distance from the current stance foot location. We convert the Cartesian distance back to polar commands for policy inputs. In the beginning of each collection episode, we reset the simulation and policy into a random state, including a randomized step command. The policy then executes for an initial number of touchdowns to settle the robot into a steady-state gait. At the 5th TD event on either the left or right foot, we record the robot state and then change to a new command. Finally, we record the subsequent touchdown with step error for that new command. This process is repeated for each step command in the 30×30 grid using the same reset state. To effectively cover a wide range of the state-command space, we can increase the data coverage by having many different reset states.

B. Model Definition & Training

The model uses high-dimensional robot states to predict the step error for a range of step commands as shown in Figure 5. We use a neural network to map between these high-dimensional spaces. Specifically, the input $\mathbf{X} \in \mathbb{R}^{42}$ is defined the same as the policy input $\mathbf{S}$. The output $\mathbf{Y} \in \mathbb{R}^{30\times30}$ represents a range of step errors spanning the command space. Each value of the output is the predicted step error from the current stance foot measured in the Cartesian distance.

One characteristics of the output is the continuity among the nearby step commands, because step commands that are next to each other can result in approximately similar touchdown behaviors in terms of step errors. Thus, we use an CNN as the network structure by applying multiple layers of 2D transposed convolution operators over the robot state. To train the model, we apply the mean-square loss over the entire grid, standard train-test split, and batch training.

V. Results

We use *cassie-mujoco-sim* simulation package[1] based on the Mujoco physics simulator [26] for the RL sampling process. Each roll-out has 300 simulation timesteps, equivalent to approximately 7.5 seconds with roughly 10 touchdowns for each foot. We empty the replay buffer at the beginning of every iteration and sample 50,000 new simulation samples before doing optimization. For optimization, each batch consists of 64 rollouts. For both the actor and critic, we use the Adam optimizer with a learning rate of 0.0005 for 5 epochs per iteration. The optimization is terminated if the KL divergence is greater than 0.02. The policy was trained for approximately 400 millions timesteps, using 56 cores on a dual Intel Xeon Platinum 8280 server on Intel vLab.

[1]https://github.com/osudrl/cassie-mujoco-sim

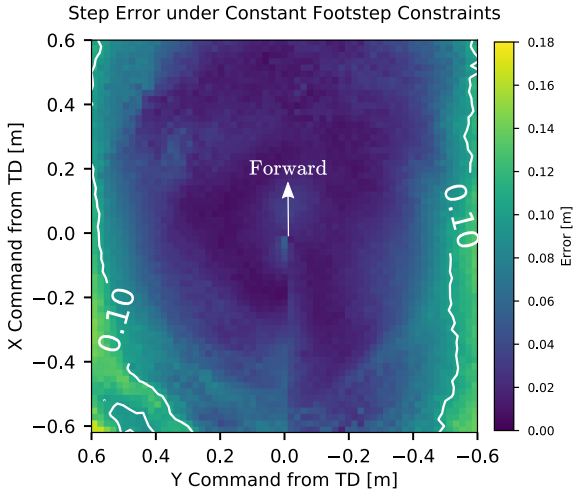


Fig. 3: Each pixel of the image shows the step error for the corresponding next same side footstep command. Commands are for the same foot that is currently in touchdown and is in relative X and Y distance away from the current touchdown location. For example, X command of 0.6 m and Y command of 0 m represents walking straight ahead, each foot having relative step length of 0.6 m and direction of 0° from the previous same side touchdown location. Step error is the Euclidean distance between the target and actual touchdown locations. We see that large side (Y command greater than 0.5m) and diagonal steps tend to have larger errors. Interestingly, due to the polar command definition, we notice the circular patterns of the contour levels.

The learned policy is first evaluated under constant footstep constraints. Then it is tested with randomized footstep commands. We show the learned dynamic gaits with sim-to-real transfer under constant and one-off footstep constraints on the bipedal robot Cassie. Lastly, we present the predicted reachability map from the learned 1-step transition model.

A. Simulation Results

We perform various experiments in simulation to validate and quantify the behaviors of the footstep policy. First, we evaluate the step accuracy when the policy is controlling Cassie with constant footstep constraints. This will mimic speed control, but also satisfies the footstep constraint every step. We sweep through the step command space within the range of step length and direction. For each step command, we record the step errors for 20 consecutive steps. As shown in Figure 3, the majority of the commands have errors less then 0.1m. The step error tends to increase with large commands. Cassie's narrow feet have no control over frontal plane motions which means large side and diagonal steps while the pelvis is facing forward are more challenging.

The second experiment intends to capture the variability needed between consecutive step commands, meaning the next footstep often has a small change from the previous one. To test the footstep behaviors under random footstep constraints, we define four categories of tests with increasing levels of difficulty. Each category has different maximum command changes from the previous footstep command. The command changes are uniformly sampled within a limit, and then added on top to the previous step command. We reject commands less than 0.01m to prevent them stuck within small footsteps. For each test, we collect 200 footsteps in simulation and track the step errors.

Additionally, we investigate briefly on the effects of step frequency onto the step accuracy in simulation. It has been previously shown that adapting the step frequency can greatly robustify locomotion [27, 28]. We experiment this by changing the phase multiplier γ explained in Section III-D. When we increase the footstep command, we would like to increase swing time (decrease γ) so the policy has more time to move the foot toward the target.

We use three different setups to test this effect. First setup (Fixed γ) is the base policy that is trained with a range of swing-to-stance ratios and step frequencies. But for this experiment, the base policy is evaluated with the nominal ratio (0.45) and step frequency ($\gamma = 1$).

Second setup (Linear γ) still uses the base policy, but changes the step frequency in a linear relationship with the command footstep length L_{step}, expressed as a remapping,

$$\gamma = \left(1 - \frac{L_{step}}{L_{step}^{max}}\right)(\gamma^{max} - \gamma^{min}) + \gamma^{min}.$$

Third setup (Heuristic γ) is a policy that is trained with a step frequency heuristics to regulate γ. Linear γ uses the one time information about the step command to vary γ. In this setup, we seek to maintain a history of information on the step command to vary γ. We keep a weighted moving average of the step commands $U^{avg} = (L^{avg}, \theta^{avg})$. Then we use the difference between the target $\Delta = ||U^{tar} - U^{avg}||_2$ to modify the phase multiple γ as the following:

$$\gamma = \begin{cases} 1.0 & \text{if } \Delta < 0.2, \\ \left(\frac{L^{avg}}{L^{tar}}\right)^2 & \text{if } \Delta \geq 0.2 \text{ and } L^{avg} \geq L^{tar} \text{ and} \\ & \quad |\theta^{avg} - \theta^{tar}| \leq \frac{\pi}{10}, \\ \frac{1}{(0.8+\Delta)^2} & \text{otherwise} \end{cases}$$

For small changes in command, there is no need to change γ so we keep it at 1. In the second case, we want to make γ greater than 1 to decrease swing time and make touchdown happen sooner. Otherwise, we make γ less than 1 to increase swing time and give the foot more time to reach the target.

From Table III, we see that step errors become larger when the difficulty of commands increases. When the step length is the same, randomizing the step direction causes the performance to drop. In general, having step frequency modifications (Linear and Heuristic γ) can improve the step accuracy. However, the hand-designed heuristic is only based on the footstep commands during training. The step frequency might also be dependent on other dynamics [24], such as energetic cost, body velocities, or momentum. We leave the smarter step frequency regulation as future work.

Range of ΔU / γ Type	$\pm 0.3m$, $\pm 0°$	$\pm 0.3m$, $\pm 20°$	$\pm 0.7m$, $\pm 0°$	$\pm 0.7m$, $\pm 20°$
Fixed	0.11±0.06m	0.14±0.08m	0.19±0.12m	0.23±0.13m
Linear	0.12±0.06m	0.15±0.09m	0.18±0.13m	0.22±0.14m
Heuristic	0.09±0.05m	0.14±0.10m	0.17±0.09m	0.17±0.10m

TABLE III: This table shows the responsiveness of randomizing footstep constraints with mean and std of the step errors. The next footstep constraints are randomized in an incremental change ΔU from the last footstep command, expressed as $U^{next} = U^{prev} + \Delta U$. By actively changing step frequency, Linear and Heuristic setup can both achieve better accuracy than constant step frequency.

B. Sim-to-Real on Cassie

The learned policies were transferred onto a bipedal robot Cassie and successfully completed multiple experiments. We used the learned policy with fixed nominal ratio and step frequency to perform the tests. First, we tested the policy under constant footstep constraints. We used the estimated feet positions and pelvis positions to calculate the relative step length of each foot on the hardware. The position data was collected using the proprietary state estimator from Agility Robotics. From Table IV, each footstep constraint is well satisfied except for larger step lengths. We noticed lateral drifting issues when the robot tried to satisfy the large step commands, causing the step length to be shorter then the desired one.

Target [m]	0.3	0.5	0.8
Measured [m]	0.32 ± 0.03	0.46 ± 0.04	0.62 ± 0.06

TABLE IV: Step command error of Cassie walking on treadmill with constant footstep constraints. Step errors are measured as the mean and the standard deviation over 50 footsteps.

Next, we tested the policy with a sequence of varying step commands. For example, when the robot is walking over gaps with different sizes, the policy should be able to regulate the footsteps. The step commands are lists of step lengths, [0.1, 0.5, 0.7, 0.4]m for the left and [0.3, 0.7, 0.5, 0.2]m for the right. Cassie starts with the left foot as shown in Figure 4.

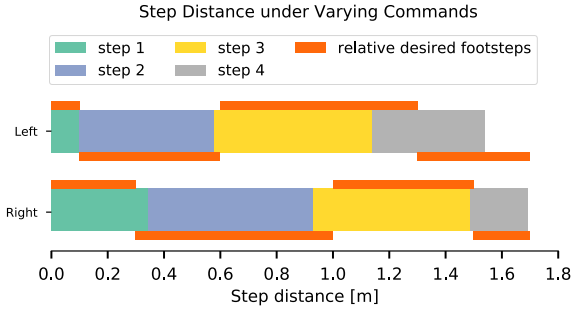


Fig. 4: On hardware, Cassie starts with stepping in place and executes a sequence of step commands for each foot, and finally goes back to stepping in place. Similar to Table IV, we noticed undershooting when the commands are larger, which leads to the left foot traveling a shorter distance overall.

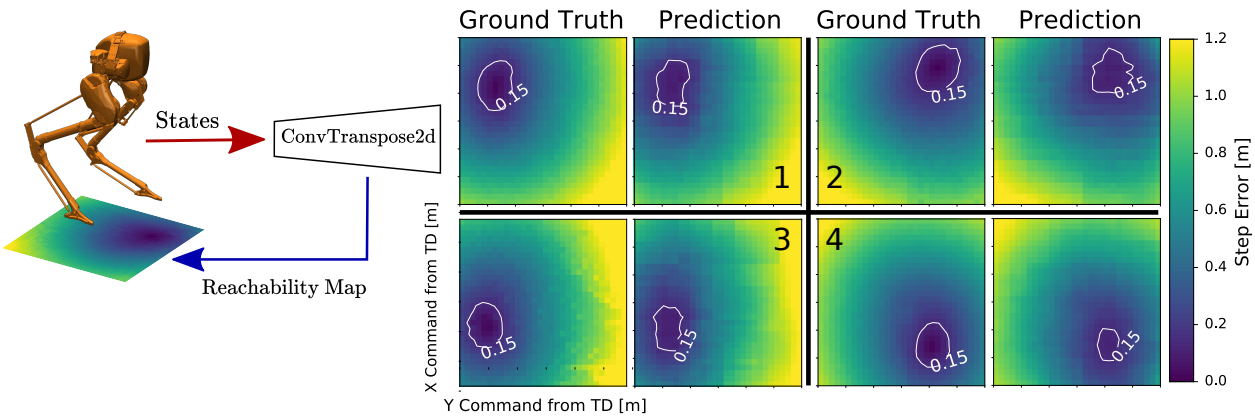


Fig. 5: 1-Step reachability map is predicted and compared with the ground truth. Four examples show that the model can accurately predict the next footstep behaviors under various robot TD states. Each case has a set of reachable footsteps. Regions within the contour lines are defined as the size of the reachable set, also plotted in Figure 6. The predicted images are smoother than ground truth in some cases (image 3), due to the image-based techniques.

Third, Cassie also performed basic 1-2 steps, where Cassie starts stepping in place, takes a desired step, and then goes back to stepping in place. We tested various of 1-2 steps maneuvers on hardware with varying directions and ground height. Cassie is able to show robust behaviors on unseen height changes when reaching to the step target. Especially when the ground becomes very uneven, the control policy will prioritize the robustness for balance over tracking footsteps. We refer readers to the accompanying video for better visual demonstrations.

C. Learned 1-Step Transition Model

We collected 9×10^6 labeled data from simulation using 10^4 reset states. We rarely found failure cases leading to the robot falling down, due to the learned policy prioritizing balance first, even though the commands can be vastly different from the previous ones. We are interested to see the size of the reachable set out of the entire command space, defined when the step errors less than 0.15m. Figure 6 shows that the policy is capable of reaching a larger range of next footsteps when the robot velocity is lower. Intuitively, when the robot travels at higher speeds, it will commit more into a narrower set of reachable TD locations, thus needing multi-step planning to resolve the transitions. Figure 5 shows four examples of the reachability map predictions under various robot states. We noticed the inductive bias of smooth variation in the output of the proposed network is potentially suitable to this application.

VI. Conclusion

In this paper, we present a learning approach for bipedal locomotion control under footstep-constraints. Our approach trains the control policy from scratch with two objectives in different timescales, periodic locomotion and sparse footstep-constraints. The control policy is conditioned on the footstep commands. The resulting learned solution shows dynamic gaits while satisfying footstep constraints imposed through user commands. We demonstrate the learned behaviors with various experiments in simulation as well as successful sim-to-real transfer on the robot Cassie.

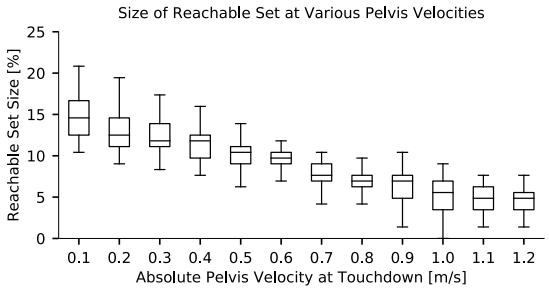


Fig. 6: From the collected data, when the robot travels at fast speed regardless of directions, the size of the reachable set (show as median and interquartile-range) becomes smaller, indicating the robot is more committed into a narrower range of commands under the learned policy. The velocities are binned into categories.

Our proposed approach can potentially lead to several future research directions by addressing and extending the following aspects. First, we observed that the robot tends to drift when inputting large steady-state step commands. We hypothesize that this is due to error in internal estimation of feet locations. We could alleviate this by directly inputting the feet positions from the estimator. Second, although we showed that the learned policy is robust to changes in step height to some extent, actively handling step height could improve previous work on blind stair walking, which treated step height as pure disturbances [8]. Third, we hope to extend the 1-step prediction model to interact with the environment by learning the reachability model for 2 to 3 steps in the future, which has been shown to be enough look ahead for planning [28]. Encoding the full-order robot state into a multi-step reachability prediction can greatly simplify the planning problem with the robust learned control policy. With the ability to actively control 3D footsteps and robustly maintain balance, a reachability map provides a common language to interact with nearby terrains and plan for multi-step behaviors, leading to reliable and robust control capable of walking up and down stairs and avoiding unsteppable regions.

Acknowledgments

We thank Intel for providing vLab resources and students at Dynamic Robotics Laboratory for helpful discussions.

REFERENCES

[1] Z. Xie, P. Clary, J. Dao, P. Morais, J. Hurst, and M. Van De Panne, "Learning Locomotion Skills for Cassie: Iterative Design and Sim-to-Real," *Conference on Robotic Learning*, no. CoRL, 2019.

[2] J. Siekmann, Y. Godse, A. Fern, and J. Hurst, "Sim-to-real learning of all common bipedal gaits via periodic reward composition," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 7309–7315.

[3] G. A. Castillo, B. Weng, W. Zhang, and A. Hereid, "Robust feedback motion policy design using reinforcement learning on a 3d digit bipedal robot," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 5136–5143.

[4] X. B. Peng, G. Berseth, K. Yin, and M. Van De Panne, "DeepLoco," *ACM Transactions on Graphics*, vol. 36, no. 4, pp. 1–13, 2017.

[5] J. Siekmann, S. Valluri, J. Dao, F. Bermillo, H. Duan, A. Fern, and J. Hurst, "Learning Memory-Based Control for Human-Scale Bipedal Locomotion," in *Proceedings of Robotics: Science and Systems*, Corvalis, Oregon, USA, July 2020.

[6] H. Duan, J. Dao, K. Green, T. Apgar, A. Fern, and J. Hurst, "Learning task space actions for bipedal locomotion," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 1276–1282.

[7] K. Green, Y. Godse, J. Dao, R. L. Hatton, A. Fern, and J. Hurst, "Learning spring mass locomotion: Guiding policies with a reduced-order model," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3926–3932, 2021.

[8] J. Siekmann, K. Green, J. Warila, A. Fern, and J. Hurst, "Blind Bipedal Stair Traversal via Sim-to-Real Reinforcement Learning," in *Proceedings of Robotics: Science and Systems*, Virtual, July 2021.

[9] V. Tsounis, M. Alge, J. Lee, F. Farshidian, and M. Hutter, "Deepgait: Planning and control of quadrupedal gaits using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3699–3706, 2020.

[10] X. Da, Z. Xie, D. Hoeller, B. Boots, A. Anandkumar, Y. Zhu, B. Babich, and A. Garg, "Learning a contact-adaptive controller for robust, efficient legged locomotion," in *4th Conference on Robot Learning, CoRL 2020, 16-18 November 2020, Virtual Event / Cambridge, MA, USA*, ser. Proceedings of Machine Learning Research, J. Kober, F. Ramos, and C. J. Tomlin, Eds., vol. 155. PMLR, 2020, pp. 883–894. [Online]. Available: https://proceedings.mlr.press/v155/da21a.html

[11] Z. Xie, H. Y. Ling, N. H. Kim, and M. Van De Panne, "ALLSTEPS: Curriculum-driven learning of stepping stone skills," *ACM SIGGRAPH/Eurographics Symposium on Computer Animation, SCA 2020*, vol. 39, no. 8, pp. 213–224, 2020.

[12] H. Geyer, A. Seyfarth, and R. Blickhan, "Compliant leg behaviour explains basic dynamics of walking and running," *Proceedings of the Royal Society B: Biological Sciences*, vol. 273, no. 1603, pp. 2861–2867, 2006.

[13] A. Wu and H. Geyer, "The 3-D spring-mass model reveals a time-based deadbeat control for highly robust running and steering in uncertain environments," *IEEE Transactions on Robotics*, vol. 29, no. 5, pp. 1114–1124, 2013.

[14] S. Kajita, F. Kanehiro, K. Kaneko, K. Yokoi, and H. Hirukawa, "The 3D linear inverted pendulum mode: A simple modeling for a biped walking pattern generation," *IEEE International Conference on Intelligent Robots and Systems*, vol. 1, pp. 239–240, 2001.

[15] W. C. Martin, A. Wu, and H. Geyer, "Experimental evaluation of deadbeat running on the ATRIAS biped," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 1085–1092, 2017.

[16] T. Apgar, P. Clary, K. Green, A. Fern, and J. Hurst, "Fast Online Trajectory Optimization for the Bipedal Robot Cassie," *Robotics: Science and Systems XIV*, 2018. [Online]. Available: http://www.roboticsproceedings.org/rss14/p54.pdf

[17] X. Xiong, J. Reher, and A. D. Ames, "Global position control on underactuated bipedal robots: Step-to-step dynamics approximation for step planning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 2825–2831.

[18] A. Meduri, M. Khadiv, and L. Righetti, "Deepq stepper: A framework for reactive dynamic walking on uneven terrain," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 2099–2105.

[19] J. Viereck and L. Righetti, "Learning a centroidal motion planner for legged locomotion," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 4905–4911.

[20] Z. Xie, X. Da, B. Babich, A. Garg, and M. van de Panne, "Glide: Generalizable quadrupedal locomotion in diverse environments with a centroidal model," *CoRR*, vol. abs/2104.09771, 2021. [Online]. Available: https://arxiv.org/abs/2104.09771

[21] S. Coros, P. Beaudoin, K. K. Yin, and M. Van De Panne, "Synthesis of constrained walking skills," *ACM SIGGRAPH Asia 2008 Papers, SIGGRAPH Asia'08*, 2008.

[22] G. Bledt and S. Kim, "Extracting Legged Locomotion Heuristics with Regularized Predictive Control," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 406–412, 2020.

[23] J. Morimoto, C. G. Atkeson, G. Endo, and G. Cheng, "Improving humanoid locomotive performance with learnt approximated dynamics via Gaussian processes for regression," *IEEE International Conference on Intelligent Robots and Systems*, pp. 4234–4240, 2007.

[24] J. E. Bertram, "Constrained optimization in human walking: Cost minimization and gait plasticity," *Journal of Experimental Biology*, vol. 208, no. 6, pp. 979–991, 2005.

[25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," pp. 1–12, 2017. [Online]. Available: http://arxiv.org/abs/1707.06347

[26] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," *IEEE International Conference on Intelligent Robots and Systems*, pp. 5026–5033, 2012.

[27] K. Nishiwaki and S. Kagami, "Strategies for adjusting the zmp reference trajectory for maintaining balance in humanoid walking," *2010 IEEE International Conference on Robotics and Automation*, pp. 4230–4236, 2010.

[28] M. Khadiv, A. Herzog, S. A. A. Moosavian, and L. Righetti, "Step timing adjustment: A step toward generating robust gaits," in *2016 IEEE-RAS 16th International Conference on Humanoid Robots (Humanoids)*, 2016, pp. 35–42.